

Tidal: Direction-Paired Buffer Pooling with a 1R1W Shared Pool

Architecture, Garnet implementation, Pool-credit policy, evaluation, limitations, and reproducibility.

1R1W Pool service	8 equal slots / pair	651 main runs	50 / 50 contribution
-----------------------------	--------------------------------	-------------------------	--------------------------------

Authors

Wang Liming (50%) · CXP-2024 (50%)

Equal-storage comparison · one-flit vnet-0 traffic · zero-delay owner-grant caveat retained

Abstract

Conventional Garnet routers assign virtual-channel (VC) storage permanently to individual input directions. This organization is simple, but it strands capacity when one side of an opposing direction pair is congested while the other side is idle. Tidal reorganizes the same physical storage into three independent direction-paired pools: East/West, North/South, and Up/Down. Each direction retains one private adaptive VC and one private escape VC in its reserved region (RES), while the pair shares four adaptive VC slots in a Pool. The resulting pair still contains eight physical slots, equal to the four-private-VC baseline.

To keep the shared storage inexpensive, the Pool is modeled with one read and one write service per pair and control virtual network per cycle. Complementary read phases allow one side to inspect RES while its peer inspects Pool. Pool credits encode slot ownership. The proposed pressure policy assigns a released credit to the direction whose RES is fuller, preserves the previous owner on a tie, and reclaims an idle peer-owned slot on demand. An owner cap of three ensures that the peer always retains at least one Pool credit.

Across 651 completed main-matrix runs, the implementation completed without deadlock and recorded no Pool read conflicts. The strongest result occurs under persistent directional skew with a long credit loop: on $8 \times 4 \times 4$, three-hop X-biased traffic, pressure cap 3 reduces network latency by 9.2% at 90/10 skew and extends the coarse stable offered-rate bracket from 0.08 to 0.10. At 100% +X above the private baseline's saturation point, it delivers 1.818× as many packets during the measurement interval. The design is not universally faster: balanced, short-link, and rapidly reversing traffic expose the cost of the fixed 1R1W phase schedule.

1. Motivation

With private input buffers, a free VC on the cold direction cannot accept a flit arriving from the hot direction. This becomes especially costly when link latency is long because a freed downstream slot is not reusable until its credit completes the return trip.

For the four-VC baseline and one-flit control traffic, four slots and an approximately 17-cycle credit loop at link latency ($L=8$) imply an analytic zero-contention input-credit limit of approximately:

Analytic input-credit limit: $4 / 17 \approx 0.24$ flits/cycle.

This limit is not always the first bottleneck. The $4 \times 4 \times 4$ tornado workload, for example, reaches an output bottleneck before the analytic input-credit limit. The design therefore needs traffic that combines a long credit loop, persistent directional imbalance, and sufficiently distributed outputs.

2. Architecture

2.1 Direction-paired storage

Each router contains three independent pairs:

- East/West;
- North/South;
- Up/Down.

For every pair and control vnet, the physical layout is:

Region	Side 0	Shared Pool	Side 1
Adaptive private VC	1	-	1
Escape VC	1	-	1
Movable adaptive VC slots	-	4	-
Total physical slots	2	4	2

Thus the proposed design and baseline both contain eight physical slots per direction pair:

Equal-storage identity: $2 \times (1 \text{ private} + 1 \text{ escape}) + 4 \text{ Pool} = 8$.

Pool membership follows the true input direction, not the selected output. For example, a flit entering from West may occupy the East/West Pool and later leave through North. Each pooled flit records its true ingress direction and upstream VC so that routing and credit return remain correct even if its physical slot resides in the peer InputUnit.

2.2 Pinned escape capacity

Escape VCs never enter the Pool and their credits never migrate. They continue to use deterministic, non-wrapping XYZ routing. Consequently, Pool ownership cannot consume or starve the deadlock-avoidance substrate.

2.3 One-read, one-write Pool

The Pool exposes at most one read and one write service per pair and control vnet per cycle.

Read access follows complementary phases:

Cycle phase	Side 0 (E/N/U)	Side 1 (W/S/D)
Even	RES	Pool
Odd	Pool	RES

Cycle parity removes cross-direction Pool-read conflicts. Each selected bank still uses ordinary VC and switch arbitration. A separate per-pair write guard allows at most one adaptive Pool allocation per cycle. If two directions arrive simultaneously, the other direction can use RES only when a reserved credit is available; otherwise it waits.

The model establishes equal buffer capacity and a 1R1W Pool service limit. It does not yet establish total chip area or energy. Owner state, selectors, control logic, and wiring require RTL synthesis before claiming zero physical cost.

3. Pool-Credit Mechanism

3.1 Credit ownership

Each of the four Pool slots has exactly one direction owner. Owning a Pool credit means that the upstream direction may reserve that slot for an incoming flit. Credits are conserved:

Credit conservation: $\text{owner_E} + \text{owner_W} = 4$.

Private adaptive and escape credits stay with their original directions.

3.2 Pressure and sticky ties

When a pooled flit leaves and releases its slot, the pressure policy compares the two RES occupancies:

- the direction with the fuller RES receives the released Pool credit;
- if occupancies tie, the previous owner keeps the credit.

The tie rule adds stickiness: a direction that recently used the Pool is likely to use it again, so ownership does not oscillate without evidence of changed demand.

3.3 Owner cap and on-demand reclaim

The recommended owner cap is three. A hot direction may own three of the four Pool credits, but the peer always retains one.

Release-time redistribution alone can freeze when an idle owner releases nothing. Therefore, if a direction needs Pool capacity and has no usable owned slot, it may immediately claim an idle peer-owned slot while remaining below the cap. Busy slots are never revoked.

The present model updates the owner table with zero additional cycles. Accordingly, the measured gains are an optimistic upper bound until an explicit one-cycle or parameterized grant path is modeled.

4. Garnet Implementation

The reproducible implementation snapshot is stored under `1R1W/overlay`. The principal changes are:

- physical-to-logical VC mapping for the pair-global Pool;
- true-ingress and upstream-VC metadata for pooled flits;
- complementary RES/Pool scheduling in `SwitchAllocator`;
- one-write-per-pair admission in the downstream allocation path;
- owner state, pressure-based release grants, cap enforcement, and on-demand reclaim in `GarnetNetwork`;
- synthetic X-biased and direction-reversal traffic;
- structural and policy instrumentation.

The implementation requires one-flit control VC buffers and adaptive Torus3D routing. Experiments use `--inj-vnet=0`, so ($L=8$) denotes eight cycles of link latency, not an eight-flit packet.

Important exported counters include Pool allocations, owned allocations, demand reclaims, owner migrations, release handoffs/keeps, write blocks, Pool reads, RES reads, Pool read conflicts, ownership-wait episodes, per-direction activity, and peak borrowed credits.

5. Evaluation

5.1 Methodology and baseline

Every comparison preserves equal physical VC capacity. The main four-VC arms are:

Arm	Physical organization per pair	Purpose
<code>private_v4</code>	4 private + 4 private	primary baseline
<code>dpphys_rts</code>	2 RES + 4 Pool + 2 RES	non-migrating purity anchor
<code>dpphys_owner_rr</code>	2 RES + 4 Pool + 2 RES	blind ownership control
<code>dpphys_pressure</code> , cap 3	2 RES + 4 Pool + 2 RES	proposed policy
<code>dpphys_pressure</code> , cap 4	2 RES + 4 Pool + 2 RES	full-elasticity upper bound

The experiments use $4 \times 4 \times 4$ or $8 \times 4 \times 4$ Torus3D networks, link latencies $L=1$ and $L=8$, one-flit vnet-0 control packets, 10,000 simulated cycles for the main long runs, and three seeds per reported point. The 651-run main matrix comprises 120 broad validation runs, 96 policy runs, 72 owner-cap runs, 192 direction-reversal runs, and 171 X-biased/VC-scale runs. A separate 48-run recovery-instrumentation rerun is not included in the 651 total.

5.2 Structural validation

All main runs completed without deadlock. Instrumented runs satisfied:

- Pool allocations = owned allocations + demand reclaims;
- owner migrations = demand reclaims + release handoffs;
- Pool reads = release handoffs + release keeps;
- per-direction allocations sum to total Pool allocations;
- owner counts sum to Pool size;
- no side exceeds its owner cap;
- Pool read conflicts = 0.

For cap 3, peak borrowing was one Pool slot, confirming that the peer retained its final credit. A dedicated simultaneous-arrival microbenchmark remains desirable for more direct write-port stress.

5.3 Persistent X-direction skew

The $8 \times 4 \times 4$ `torus3d_xbiased` workload sends each packet three minimal hops in X and varies the probability of choosing +X. This keeps traffic spatially distributed while controlling directional imbalance.

At ($L=8$), injection 0.10, and four physical VCs:

+X probability	Cap-3 throughput/base	Cap-3 latency/base
50%	1.000	1.023
70%	1.000	1.016
80%	1.000	1.001
90%	1.021	0.908
100%	1.818	0.280

At 50-70% +X, the design pays a 1.6-2.3% phase penalty because little cold-side capacity is stranded. It breaks even near 80/20. At 90/10, cap 3 reduces network latency by 9.2%. The 100% point is above private-v4 saturation, so its latency ratio is not a valid below-saturation comparison; the meaningful result is 1.818× delivered throughput.

Using completion/acceptance near one as a coarse stability criterion, the highest tested stable offered rates are approximately 0.08 for private-v4, 0.10 for cap 3, and 0.12 for cap 4. These are brackets, not exact saturation measurements.

5.4 Tornado and policy behavior

On 4×4×4, `torus3d_tornado` moves each packet +1 in X, +1 in Y, and +1 in Z. It therefore keeps West, South, and Down inputs hot rather than loading only one dimension.

At (L=8), injection 0.9:

Mode	Throughput/base	Network latency	Latency/base
private-v4	1.000	49.37	1.000
RTS	0.994	58.25	1.180
owner-RR	0.435	157.03	3.181
pressure cap 3	1.000	46.97	0.952

Pressure cap 3 reduces latency by 4.85% without materially changing throughput because another output bottleneck limits the workload. Only about 0.49% of Pool allocations require on-demand reclaim, showing that the useful behavior is mostly stable ownership with occasional demand-driven movement.

Blind owner-RR performs poorly because release-time rotation can park credits at an inactive peer and provides no demand-driven reclaim. This result concerns Pool-credit ownership, not the round-robin arbitration used inside the read-bank scheduler.

5.5 Limits and negative cases

- Uniform traffic at (L=8), injection 0.9 increases latency by approximately 3.2%.
- X-opposite traffic raises latency by approximately 3-15% because both sides contend for a single Pool.
- Tornado at (L=1) increases latency by approximately 25.1%; the credit loop is too short for extra capacity to repay the phase cost.
- Reversing the hot X direction every 16 cycles increases latency by 7.8%; ownership changes while old traffic is still draining.

These results position Tidal as a specialization for persistent directional skew near the private design's input-credit saturation knee, not as a universal replacement for private buffering.

6. Division of Labor

The project was completed collaboratively, with an overall contribution split of **50% for Wang Liming and 50% for CXP-2024**. Both contributors participated in architecture discussions, implementation review, debugging, evaluation, interpretation, report writing, and slide preparation. The responsibilities below identify the areas each person primarily led; they do not represent isolated work packages.

Wang Liming - 50%

- proposed the original direction-paired Pool concept and helped refine the final research question;
- implemented and tested the earlier 1R2W organization, providing the design experience that motivated the lower-port 1R1W version;
- developed traffic-pattern coverage and completed major parts of the evaluation matrix;
- reviewed the 1R1W behavior, Pool-credit policy results, and correctness evidence;
- co-analyzed the positive and negative results;
- co-authored and revised the report, slides, and presentation narrative.

CXP-2024 - 50%

- co-developed the direction-paired architecture and finalized the 1R1W complementary-phase organization;
- designed the Pool-credit ownership mechanism, including pressure grants, sticky ties, owner cap 3, and on-demand reclaim;
- implemented the final 1R1W mechanism and instrumentation in Garnet;
- ran and analyzed major parts of the 1R1W evaluation and cross-checked the reported data;
- co-analyzed the positive and negative results;
- co-authored and revised the report, slides, and presentation narrative.

Shared Deliverables

The following deliverables have joint 50/50 ownership: architecture decisions, debugging and correctness checks, experiment review, result interpretation, report integration, slide production, and presentation preparation.

7. Limitations and Next Steps

The current results are preliminary simulation evidence. The highest-priority next steps are:

- 1 model a one-cycle or parameterized Pool-owner grant path;
- 2 run longer, drain-aware, five-seed sweeps with 0.005 injection steps around each saturation knee;
- 3 add a directed simultaneous E/W Pool-write microbenchmark;
- 4 evaluate multi-flit packets after defining packet/VC-level ownership retention;
- 5 synthesize an RTL representation of the Pool, owner table, selectors, and control path to quantify area and energy;
- 6 compare 1R1W against an idealized dual-read Pool to isolate the cost of the single-read decision.

8. Reproducibility

The exact 1R1W snapshot and reconstruction procedure are documented in [1R1W/README.md](#). The experiment runner, analyzer, raw CSV files, and detailed reports are retained in the snapshot. The current repository's main source tree should not be overwritten when reproducing the archived implementation; reconstruct it in a temporary Git worktree as described in that README.